



# Chapter 15: Query Processing

**Database System Concepts, 7<sup>th</sup> Ed.**

©Silberschatz, Korth and Sudarshan  
See [www.db-book.com](http://www.db-book.com) for conditions on re-use



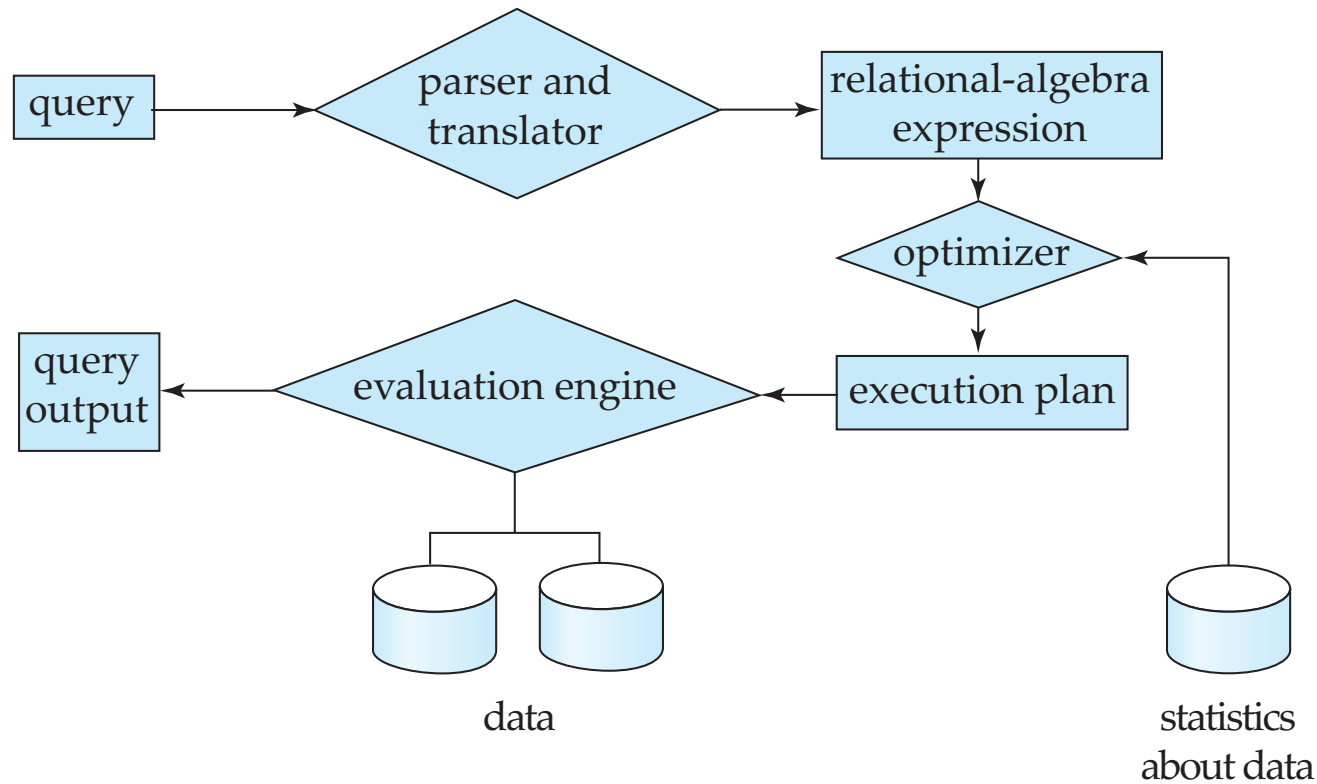
# Chapter 15: Query Processing

- Overview
- Measures of Query Cost
- Selection Operation
- Join Operation
- Other Operations
- Evaluation of Expressions



# Basic Steps in Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation





# Basic Steps in Query Processing (Cont.)

- Parsing and translation
  - translate the query into its internal form. This is then translated into relational algebra.
  - Parser checks syntax, verifies relations
- Evaluation
  - The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.



# Basic Steps in Query Processing: Optimization

- A relational algebra expression may have many equivalent expressions
  - E.g.,  $\sigma_{salary < 75000}(\Pi_{salary}(instructor))$  is equivalent to  $\Pi_{salary}(\sigma_{salary < 75000}(instructor))$
- Each relational algebra operation can be evaluated using one of several different algorithms
  - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan**. E.g.,:
  - Use an index on *salary* to find instructors with salary < 75000,
  - Or perform complete relation scan and discard instructors with salary  $\geq 75000$



# Basic Steps: Optimization (Cont.)

- **Query Optimization:** Amongst all equivalent evaluation plans choose the one with lowest cost.
  - Cost is estimated using statistical information from the database catalog
    - e.g.. number of tuples in each relation, size of tuples, etc.
- In this chapter we study
  - How to measure query costs
  - Algorithms for evaluating relational algebra operations
  - How to combine algorithms for individual operations in order to evaluate a complete expression



# Measures of Query Cost

- Many factors contribute to time cost
  - *disk access, CPU, and network communication*
- Cost can be measured based on
  - **response time**, i.e. total elapsed time for answering query, or
  - **total resource consumption**
- We use **total resource consumption** as cost metric
  - Response time harder to estimate, and minimizing resource consumption is a good idea in a shared database
- We ignore CPU costs for simplicity
  - Real systems do take CPU cost into account
  - Network costs must be considered for parallel systems
- We describe how to estimate the cost of each operation
  - We do not include cost to writing output to disk



# Measures of Query Cost

- Disk cost can be estimated as:
  - Number of seeks \* average-seek-cost
  - Number of blocks read \* average-block-read-cost
  - Number of blocks written \* average-block-write-cost
- For simplicity we just use the **number of block transfers** (*from disk to memory*) and the **number of seeks** as the cost measures
  - $t_T$  – time to transfer one block
    - Assuming for simplicity that write cost is same as read cost
  - $t_S$  – time for one seek
  - Cost for b block transfers plus S seeks
$$b * t_T + S * t_S$$
- $t_S$  and  $t_T$  depend on where data is stored; with 4 KB blocks:
  - High end magnetic disk:  $t_S = 4$  msec and  $t_T = 0.1$  msec
  - SSD:  $t_S = 20$ -90 microsec and  $t_T = 2$ -10 microsec for 4KB

**Seek time** – time to reposition the arm over the correct track of the hard disk.





## Measures of Query Cost (Cont.)

- Required data may be buffer resident already, avoiding disk I/O
  - But hard to take into account for cost estimation
- Several algorithms can reduce disk IO by using extra buffer space
  - Amount of real memory available to buffer depends on other concurrent queries and OS processes, known only during execution
- Worst case estimates assume that no data is initially in buffer and only the minimum amount of memory needed for the operation is available
  - But more optimistic estimates are used in practice



# Selection Operation

- **File scan**
- Algorithm **A1 (linear search)**. Scan each file block and test all records to see whether they satisfy the selection condition.
  - Cost estimate =  $b_r$  block transfers + 1 seek
    - $b_r$  denotes number of blocks containing records from relation  $r$
  - If selection is on a key attribute, can stop upon having found record
    - cost =  $(b_r/2)$  block transfers + 1 seek
  - Linear search can be applied regardless of:
    - selection condition or
    - ordering of records in the file, or
    - availability of indices



# Selections Using Indices

- **Index scan** – search algorithms that use an index
  - selection condition must be on search-key of index.
  
- **A2 (clustering index, equality on candidate key)**. Retrieve a single record that satisfies the corresponding equality condition
  - $Cost = (h_i + 1) * (t_T + t_S)$
  - $h_i$  : the height of the B+-tree
  - Index lookup traverses the height of the tree, plus one I/O to fetch the record;
  - each of these I/O operations requires a seek and a block transfer.



# Selections Using Indices

- **A3 (clustering index, equality on non-key)** Retrieve multiple records.
  - Records will be on consecutive blocks
  - One seek for each level of the tree, one seek for the first block.
  - Let  $b$  = number of blocks containing matching records with the specified search key, all of which are read.
  - $Cost = h_i * (t_T + t_S) + t_S + t_T * b$
- **A4 (secondary index, equality on key/non-key).**
  - On key: Retrieve a single record if the search-key is a candidate key
    - $Cost = (h_i + 1) * (t_T + t_S)$
  - On nonkey: Retrieve multiple records if search-key is not candidate key
    - each of  $n$  matching records may be on a different block
    - $Cost = (h_i + n) * (t_T + t_S)$ 
      - Can be very expensive!



# Selections Involving Comparisons

- Can implement selections of the form  $\sigma_{A \leq V}(r)$  or  $\sigma_{A \geq V}(r)$  by using
  - a linear file scan,
  - or by using indices in the following ways:
- **A5 (clustering index, comparison).** (Relation is sorted on A)
  - For  $\sigma_{A \geq V}(r)$  use the index to find first tuple  $\geq v$  and scan relation sequentially from there. The cost estimate is identical to that for case A3:  $\text{Cost} = h_i * (t_T + t_S) + t_S + t_T * b$
  - For  $\sigma_{A \leq V}(r)$  just scan relation (leaf nodes) sequentially till first tuple where  $A > v$ ; index look-up not required.
- **A6 (non-clustering index, comparison).**
  - For  $\sigma_{A \geq V}(r)$  use index to find first index entry  $A \geq v$  and scan index sequentially from there, to find pointers to records.
  - For  $\sigma_{A \leq V}(r)$  just scan leaf pages of index to find the pointers to records, till first entry  $A > v$
  - In either case, retrieve records that are pointed to
  - Cost:  $(h_i + n) * (t_T + t_S)$ , identical to the case of A4, equality on non-key.
  - requires an I/O per record; Linear file scan may be cheaper!



# Implementation of Complex Selections

- **Conjunction:**  $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$
- **A7 (conjunctive selection using one index).**
  - Select a combination of  $\theta_i$  and algorithms A1 through A6 that results in the least cost for  $\sigma_{\theta_i}(r)$ .
  - Test other conditions on tuple after fetching it into memory buffer.
- **A8 (conjunctive selection using composite index).**
  - Use appropriate composite (multiple-key) index if available.
  - The type of index determines which of algorithms A2, A3, or A4 will be used.
- **A9 (conjunctive selection by intersection of identifiers).**
  - Requires indices with record pointers.
  - Use corresponding index for each condition, and take intersection of all the obtained sets of record pointers. Then fetch records from file.
  - The cost is the sum of the costs of individual index scans, plus the cost of retrieving the records in the intersection of the retrieved lists of pointers.
  - If some conditions do not have appropriate indices, apply test in memory.



# Algorithms for Complex Selections

- **Disjunction:**  $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$ .
- **A10 (disjunctive selection by union of identifiers).**
  - Applicable if *all* conditions have available indices.
    - Otherwise use linear scan.
  - Use corresponding index for each condition, and take union of all the obtained sets of record pointers.
  - Then fetch records from file
- **Negation:**  $\sigma_{\neg\theta}(r)$ 
  - Use linear scan on file
  - If very few records satisfy  $\neg\theta$ , and an index is applicable to  $\theta$ 
    - Find satisfying records using index and fetch from file



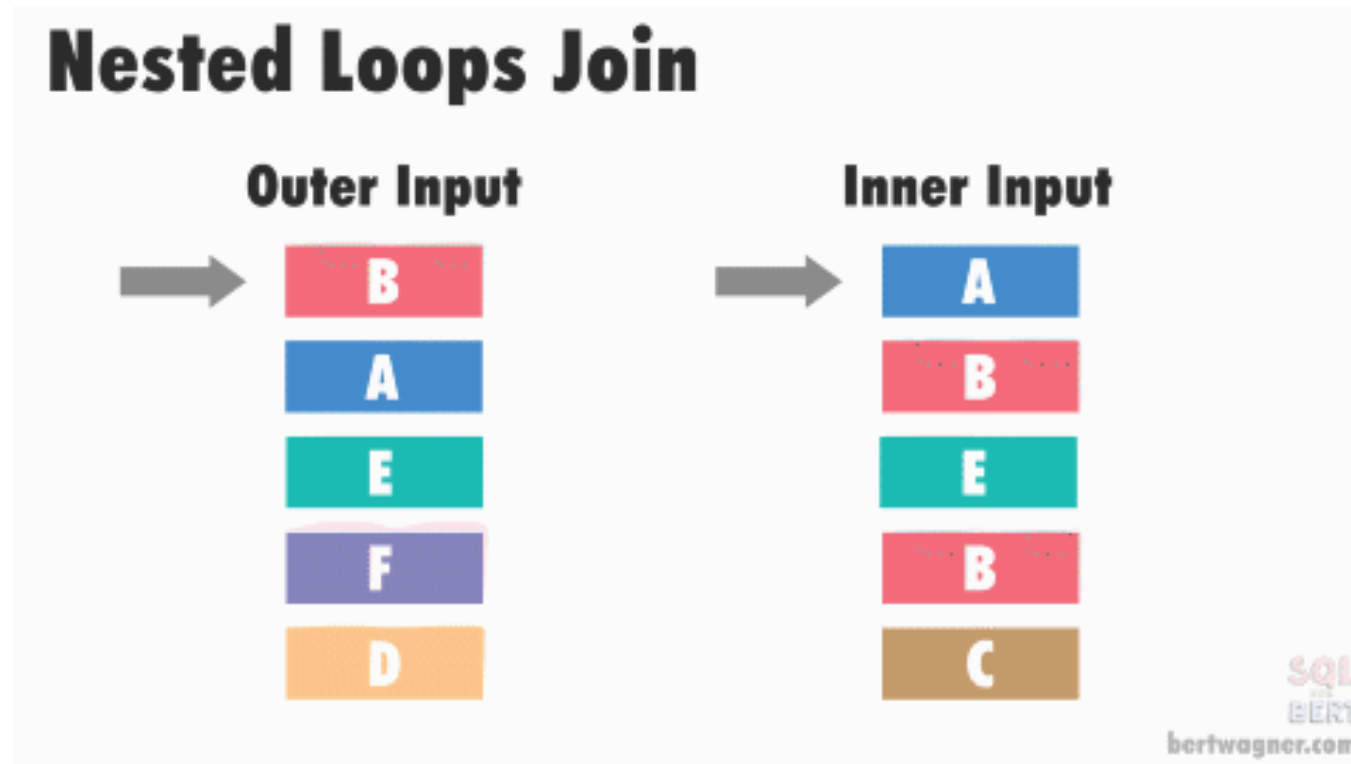
# Join Operation

- Several different algorithms to implement joins
  - Nested-loop join
  - Block nested-loop join
  - Indexed nested-loop join
- Choice based on cost estimate
- Examples use the following information
  - Number of records in *student*: 5,000      *takes*: 10,000
  - Number of blocks for *student*: 100      *takes*: 400





# Nested-Loop Join





# Nested-Loop Join

- To compute the theta join  $r \bowtie_{\theta} s$   
    **for each** tuple  $t_r$  **in**  $r$  **do begin**  
        **for each** tuple  $t_s$  **in**  $s$  **do begin**  
            test pair  $(t_r, t_s)$  to see if they satisfy the join condition  $\theta$   
            if they do, add  $t_r \cdot t_s$  to the result.  
        **end**  
    **end**
- $r$  is called the **outer relation** and  $s$  the **inner relation** of the join.
- Requires no indices and can be used with any kind of join condition.
- Expensive since it examines every pair of tuples in the two relations.



## Nested-Loop Join (Cont.)

- In the worst case, if there is enough memory only to hold one block of each relation, the estimated cost is
  - $n_r * b_s + b_r$  block transfers
  - $n_r + b_r$  seeks:  $b_r$  seeks to read  $r$ , then one seek for each scan on the inner relation  $s$
- Assuming worst case memory availability cost estimate is
  - with *student* as outer relation:

$$\begin{array}{ll} n_{student} = 5,000 & n_{takes} = 10,000 \\ b_{student} = 100 & b_{takes} = 400 \end{array}$$

    - $5000 * 400 + 100 = 2,000,100$  block transfers,
    - $5000 + 100 = 5100$  seeks
  - with *takes* as the outer relation
    - $10000 * 100 + 400 = 1,000,400$  block transfers and 10,400 seeks
- If the smaller relation fits entirely in memory, use it as the inner relation.
  - Reduces cost to  $b_r + b_s$  block transfers and 2 seeks
- If smaller relation (*student*) fits entirely in memory, the cost estimate will be 500 block transfers.
- Block nested-loops algorithm (next slide) is preferable.



# Block Nested-Loop Join

- Variant of nested-loop join in which every block of inner relation is paired with every block of outer relation.

```
for each block  $B_r$  of  $r$  do begin  
  for each block  $B_s$  of  $s$  do begin  
    for each tuple  $t_r$  in  $B_r$  do begin  
      for each tuple  $t_s$  in  $B_s$  do begin  
        Check if  $(t_r, t_s)$  satisfy the join condition  
        if they do, add  $t_r \cdot t_s$  to the result.  
      end  
    end  
  end  
end
```



## Block Nested-Loop Join (Cont.)

- Worst case estimate:
  - Each block in the inner relation  $s$  is read once for each *block* in the outer relation  $r$
  - $b_r * b_s + b_r$  block transfers: reads  $b_r$  blocks of  $r$ ; then for each block of  $r$ , reads  $b_s$  blocks of  $s$
  - $2 * b_r$  seeks: 1 seek to read one block of  $r$ , totaling  $b_r$  seeks; for each block of  $r$ , 1 seek to locate the blocks of  $s$ , totaling  $b_r$  seeks; so all together  $b_r + b_r$  seeks
- Best case:  $b_r + b_s$  block transfers + 2 seeks.



# Indexed Nested-Loop Join

- Index lookups can replace file scans if
  - join is an equi-join and natural join
  - an index is available on the inner relation's join attribute
    - Can construct an index just to compute a join.
- For each tuple  $t_r$  in the outer relation  $r$ , use the index to look up tuples in  $s$  that satisfy the join condition with tuple  $t_r$ .
- Worst case: buffer has space for only one block of  $r$ , and, for each tuple in  $r$ , we perform an index lookup on  $s$ .
- Cost of the join:  $b_r (t_T + t_S) + n_r * c$ 
  - Where  $c$  is the cost of traversing index and fetching all matching  $s$  tuples for one tuple of  $r$
  - $c$  can be estimated as cost of a single selection on  $s$  using the join condition.
- If indices are available on join attributes of both  $r$  and  $s$ , use the relation with fewer tuples as the outer relation.



# Example of Nested-Loop Join Costs

- Compute  $student \bowtie takes$ , with ***student* as the outer relation**.
- Let *takes* have a primary B<sup>+</sup>-tree index on the attribute *ID*, which contains 20 entries in each index node.
- Since *takes* has 10,000 tuples, the height of the tree is 4, and one more access is needed to find the actual data
- *student* has 5000 tuples
- Cost of block nested loops join
  - $400 * 100 + 100 = 40,100$  block transfers +  $2 * 100 = 200$  seeks
    - assuming worst case memory
    - may be significantly less with more memory
- Cost of indexed nested loops join
  - $100 + 5000 * 5 = 25,100$  block transfers and seeks.
  - CPU cost likely to be less than that for block nested loops join

|                       |                      |
|-----------------------|----------------------|
| $n_{student} = 5,000$ | $n_{takes} = 10,000$ |
| $r_{student} = 100$   | $r_{takes} = 400$    |



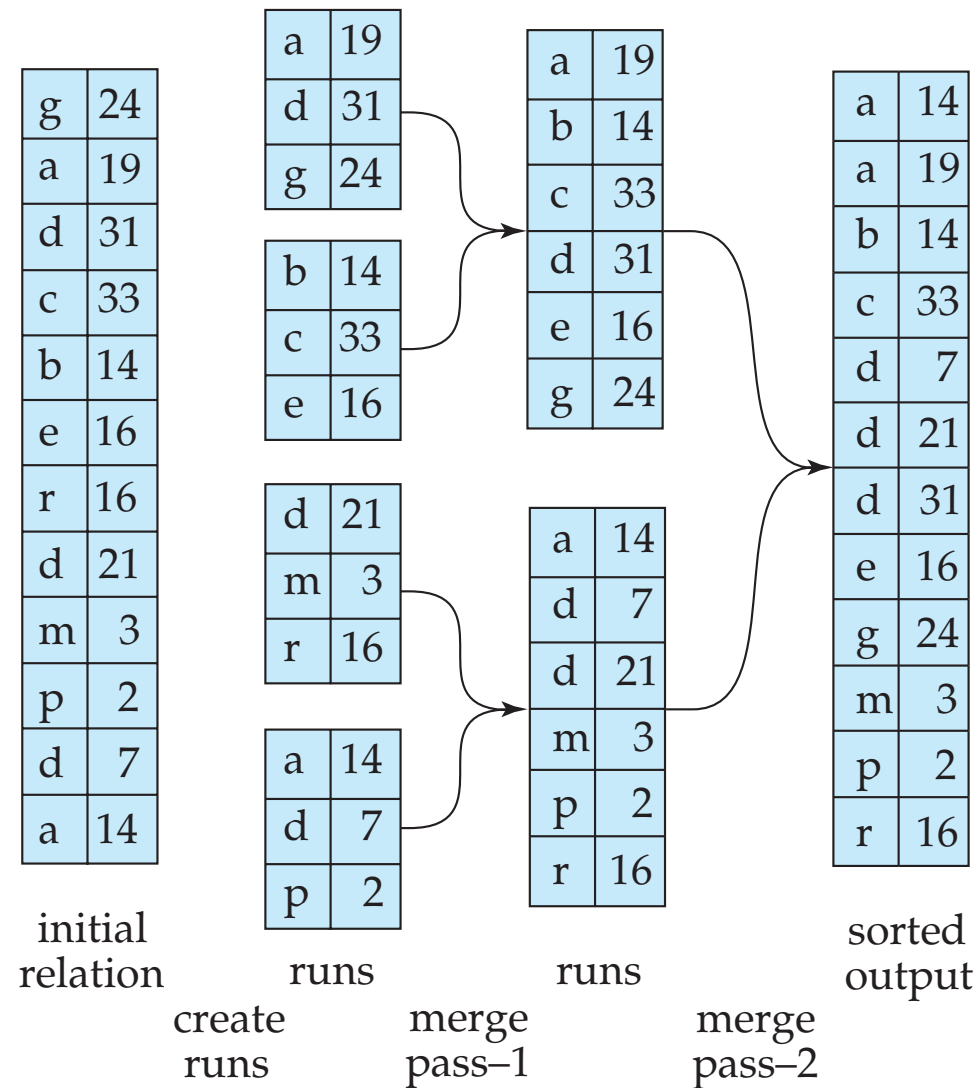
# Sorting

- We may build an index on the relation, and then use the index to read the relation in sorted order. May lead to one disk block access for each tuple.
- For relations that fit in memory, techniques like quicksort can be used.
- For relations that don't fit in memory, **external sort-merge** is a good choice.





# Example: External Sorting Using Sort-Merge





# External Sort-Merge

Let  $M$  denote memory size (the number of blocks in the main memory buffer available for sorting). Suppose  $M=3$ .

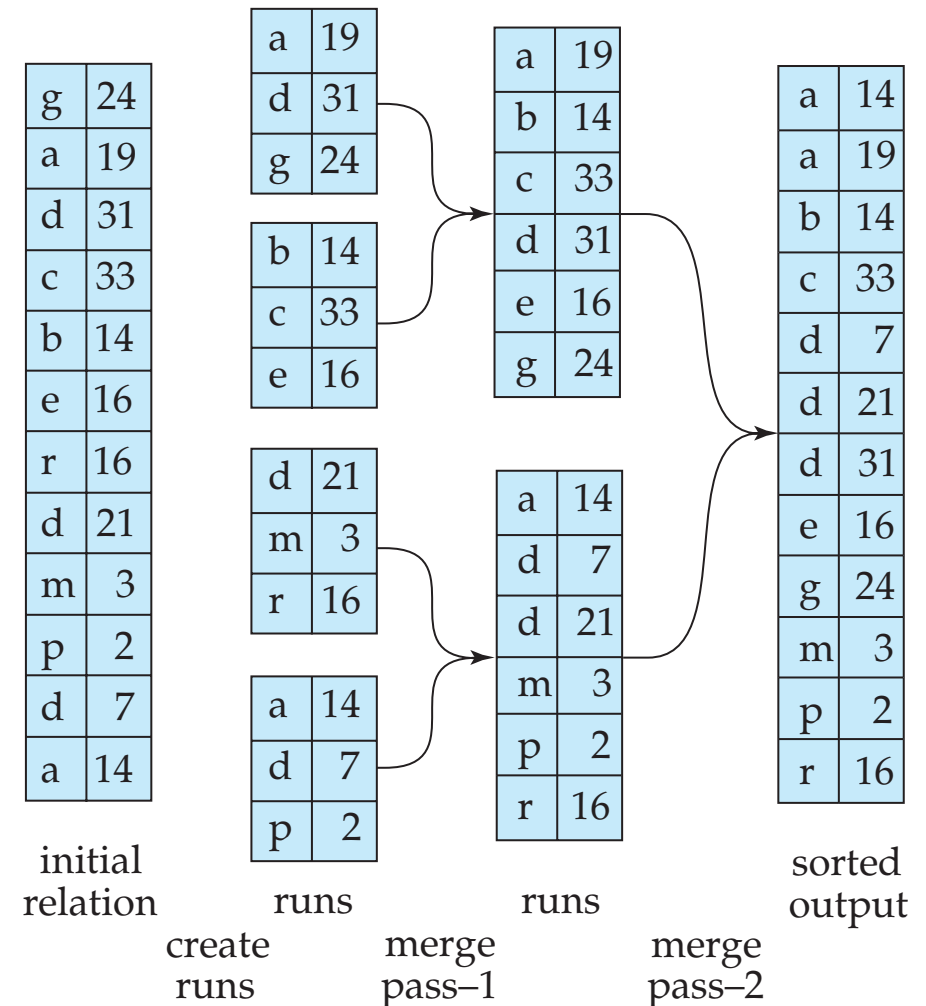
1. **Create sorted runs.** Let  $i$  be 0 initially.

Repeatedly do the following till the end of the relation:

- Read  $M$  blocks of relation into memory
- Sort the in-memory blocks
- Write sorted data to run  $R_i$ ; increment  $i$ .

Let the final value of  $i$  be  $N$  (4 sorted runs in this case)

2. *Merge the runs (next slide).....*





# External Sort-Merge (Cont.)

**2. Merge the runs (N-way merge).** We assume (for now) that  $N < M$ . Suppose  $N = 2$ .

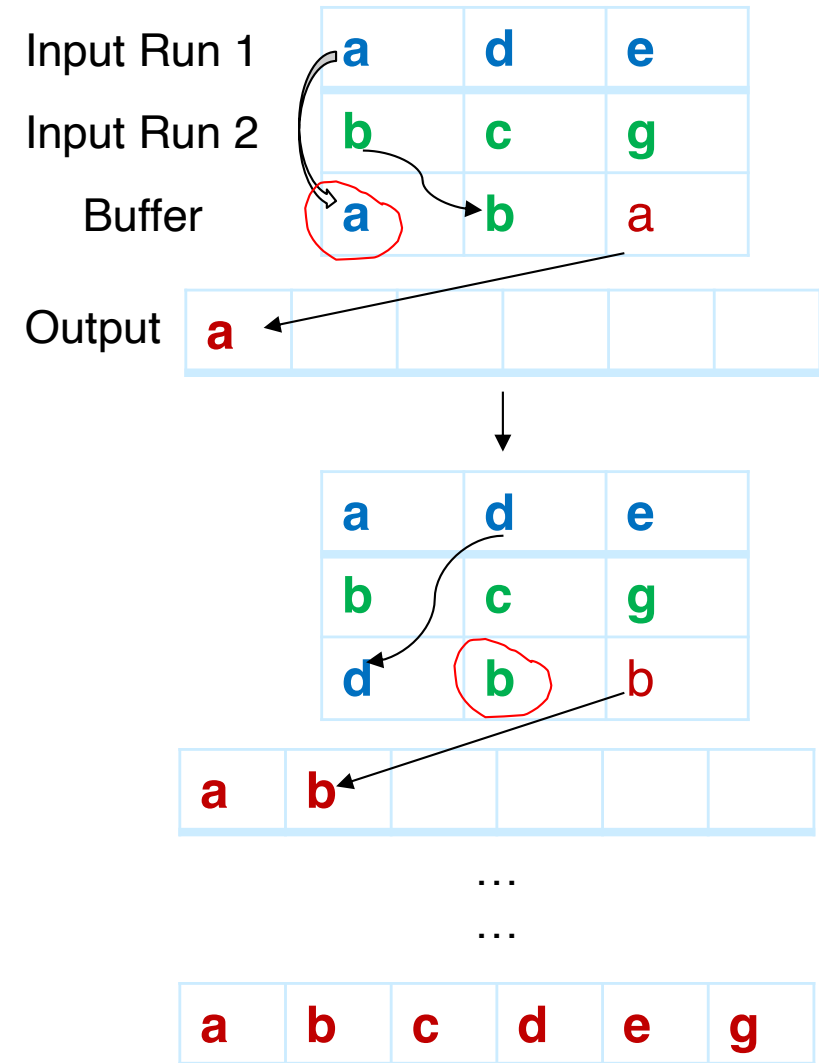
**1.** Use  $N$  blocks of memory to buffer input runs, and 1 block to buffer output. Read the first block of each run into its buffer page

**2. repeat**

- 1.** Select the first record (in sort order) among all buffer pages
- 2.** Write the record to the output buffer. If the output buffer is full write it to disk.
- 3.** Delete the record from its input buffer page.

**If** the buffer page becomes empty **then** read the next block (if any) of the run into the buffer.

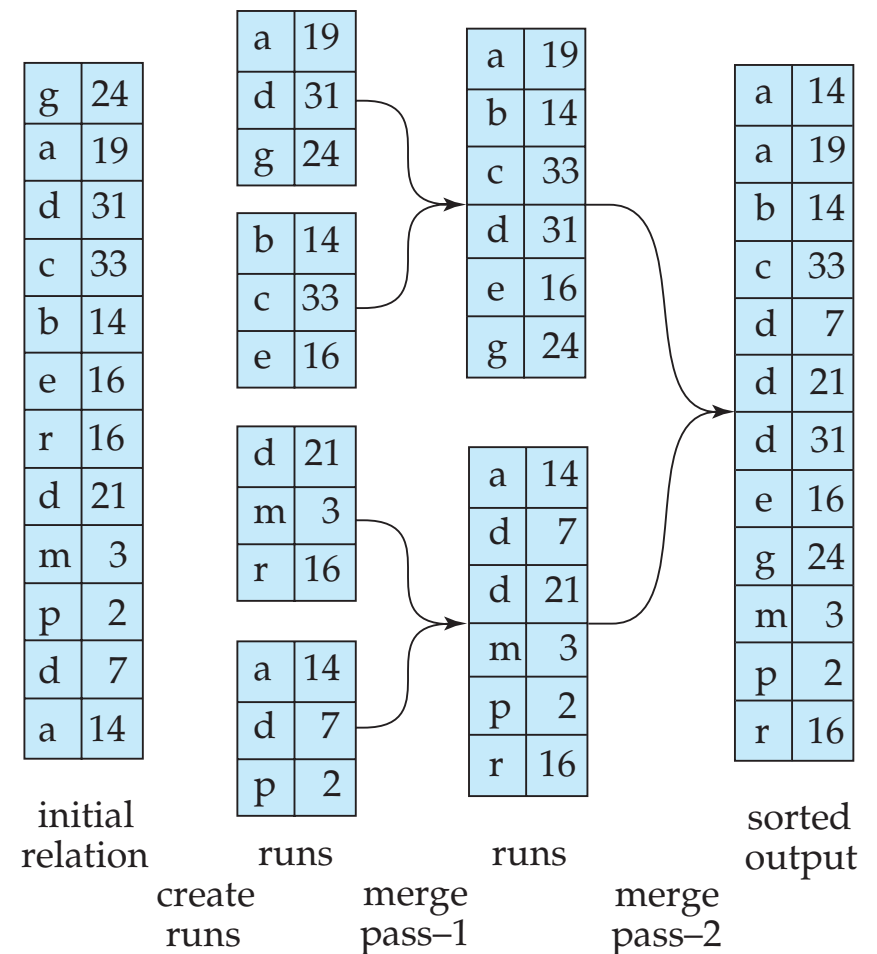
**3. until** all input buffer pages are empty:





## External Sort-Merge (Cont.)

- If  $N \geq M$ , several merge *passes* are required.
  - In each pass, contiguous groups of  $M - 1$  runs are merged, i.e.  $(M-1)$ -way merge.
  - A pass reduces the number of runs by a factor of  $M - 1$ , and creates runs longer by the same factor.
    - E.g. If  $M=3$  and there are 4 runs, one pass reduces the number of runs to 2
  - Repeated passes are performed till all runs have been merged into one.





# External Merge Sort (Cont.)

## ■ Cost analysis:

- $b_r$  denotes the number of blocks containing records of relation  $r$ .
- **The initial run creation stage:**
  - $2b_r$  block transfers (reading every block and writing them out again)
  - The initial number of runs is  $\lceil b_r/M \rceil$
- **The merge stage:**
  - 1 block per run leads to too many seeks during merge
  - Instead, use  $b_b$  buffer blocks per run  $\rightarrow$  read/write  $b_b$  blocks at a time
  - Can merge  $\lfloor M/b_b \rfloor - 1$  runs in one pass
    - E.g.,  $M=6$ ,  $b_b=2$

Input Run 1

Input Run 2

Buffer  
( $M=6$ )

|   |   |   |
|---|---|---|
| a | d | e |
| b | c | g |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| a | d | b | c | a | b |
|---|---|---|---|---|---|

- Total number of merge passes required:  $\lceil \log_{\lfloor M/b_b \rfloor - 1} (b_r/M) \rceil$ .



## External Merge Sort (Cont.)

- Cost analysis:
  - Total number of merge passes required:  $\lceil \log_{\lfloor M/bb \rfloor - 1} (b_r/M) \rceil$ .
  - Block transfers for initial run creation as well as in each pass is  $2b_r$ 
    - for final pass, we don't count write cost
      - we ignore final write cost for all operations since the output of an operation may be sent to the parent operation without being written to disk
    - Thus total number of block transfers for external sorting:
      - $2b_r + 2b_r \lceil \log_{\lfloor M/bb \rfloor - 1} (b_r/M) \rceil - b_r$   
 $= b_r (2 \lceil \log_{\lfloor M/bb \rfloor - 1} (b_r/M) \rceil + 1)$



## External Merge Sort (Cont.)

- Cost of seeks
  - During run generation: one seek to read each run and one seek to write each run
    - $2 \lceil b_r / M \rceil$
  - During the merge phase
    - Need  $2 \lceil b_r / b_b \rceil$  seeks for each merge pass
      - except the final one which does not require a write
    - Total number of seeks:  
$$2 \lceil b_r / M \rceil + \lceil b_r / b_b \rceil (2 \lceil \log_{\lfloor M/b_b \rfloor} (b_r / M) \rceil - 1)$$



# Merge-Join

1. Sort both relations on their join attribute (if not already sorted).
2. Merge the sorted relations to join them
  1. Join step is similar to the merge stage of the sort-merge algorithm.
  2. Main difference is the handling of duplicate values in join attribute: every pair with same value on join attribute must be matched

*pr* →

| <i>a1</i> | <i>a2</i> |
|-----------|-----------|
| a         | 3         |
| b         | 1         |
| d         | 8         |
| d         | 13        |
| f         | 7         |
| m         | 5         |
| q         | 6         |

*r*

*ps* →

| <i>a1</i> | <i>a3</i> |
|-----------|-----------|
| a         | A         |
| b         | G         |
| c         | L         |
| d         | N         |
| m         | B         |

*s*

|               |                |
|---------------|----------------|
| Buffer: (a,a) | Results: (a,a) |
| Buffer: (b,b) | (b,b)          |
| Buffer: (d,d) | (d,d)          |
| Buffer: (d,d) | (d,d)          |
|               | .....          |





## Merge-Join (Cont.)

- Can be used only for equi-joins and natural joins
- Each block needs to be read only once (assuming all tuples for any given value of the join attributes fit in memory)
- Thus the cost of merge join is:  
 $b_r + b_s$  block transfers  $+ \lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil$  seeks  
+ the cost of sorting if relations are unsorted.



# Hash-Join

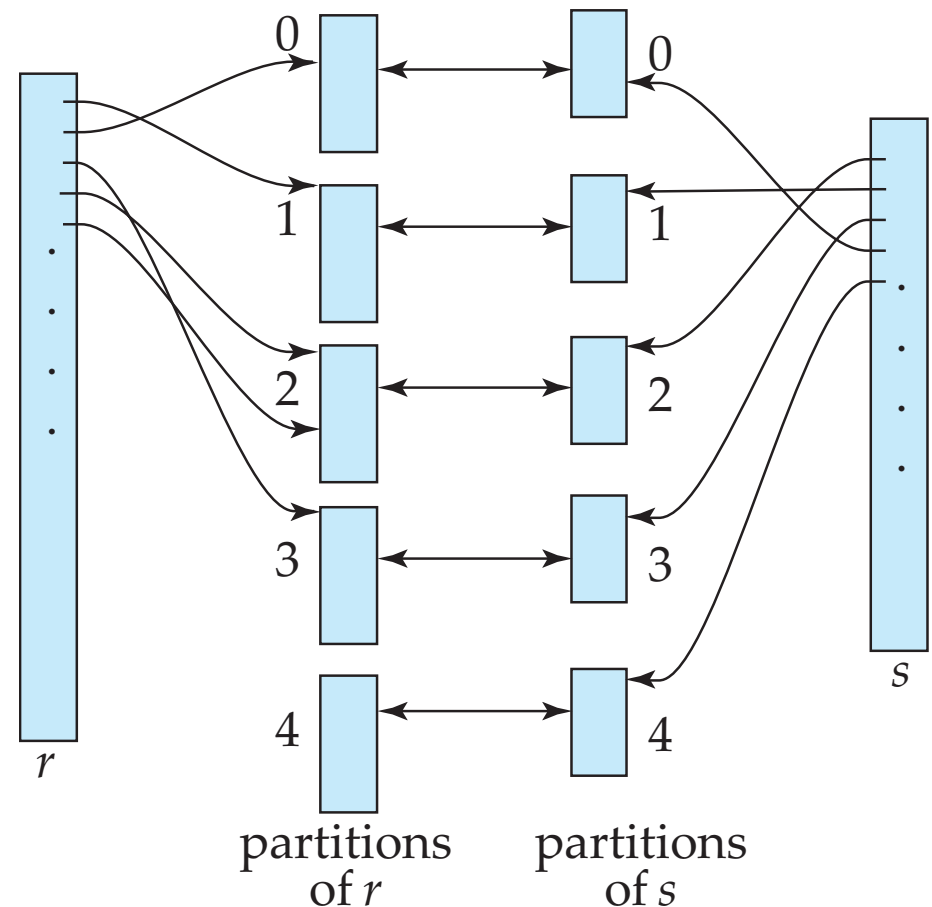
- Applicable for equi-joins and natural joins.
- A hash function  $h$  is used to partition tuples of both relations
- $h$  maps  $JoinAttrs$  values to  $\{0, 1, \dots, n\}$ , where  $JoinAttrs$  denotes the common attributes of  $r$  and  $s$  used in the natural join.
  - $r_0, r_1, \dots, r_n$  denote partitions of  $r$  tuples
    - Each tuple  $t_r \in r$  is put in partition  $r_i$  where  $i = h(t_r[JoinAttrs])$ .
  - $s_0, s_1, \dots, s_n$  denotes partitions of  $s$  tuples
    - Each tuple  $t_s \in s$  is put in partition  $s_i$ , where  $i = h(t_s[JoinAttrs])$ .



## Hash-Join (Cont.)

$r$  tuples in  $r_i$  need only to be compared with  $s$  tuples in  $s_i$ , but need not be compared with  $s$  tuples in any other partition, since:

- an  $r$  tuple and an  $s$  tuple that satisfy the join condition will have the same hash value for the join attributes.
- If that value is hashed to some value  $i$ , the  $r$  tuple has to be in  $r_i$  and the  $s$  tuple in  $s_i$ .





# Hash-Join Algorithm

The hash-join of  $r$  and  $s$  is computed as follows.

1. Partition the relation  $s$  using hashing function  $h$ . When partitioning a relation, one block of memory is reserved as the output buffer for each partition.
2. Partition  $r$  similarly.
3. For each  $i$ :
  - (a) Load  $s_i$  into memory and build an in-memory hash index on it using the join attribute. This hash index uses a different hash function than the earlier one  $h$ .
  - (b) Read the tuples in  $r_i$  from the disk one by one. For each tuple  $t_r$ , locate each matching tuple  $t_s$  in  $s_i$  using the in-memory hash index. Output the concatenation of their attributes.

Relation  $s$  (*smaller*) is called the **build input**, and  $r$  (*larger*) is called the **probe input**.



## Hash-Join algorithm (Cont.)

- The value  $n$  (i.e., *number of partitions*) and the hash function  $h$  is chosen such that each  $s_i$  should fit in memory.
  - Typically  $n$  is chosen as  $\lceil b_s/M \rceil * f$  where  $f$  is a “**fudge factor**”, typically around 1.2
  - The probe relation partitions  $h_i$  need not fit in memory
- **Recursive partitioning** required if number of partitions  $n$  is greater than number of pages  $M$  of memory.
  - instead of partitioning  $n$  ways, use  $M - 1$  partitions for  $s$
  - Further partition the  $M - 1$  partitions using a different hash function
  - Use same partitioning method on  $r$
  - Rarely required:
    - e.g., with block size of 4 KB, recursive partitioning not needed for relations of  $< 1\text{GB}$  with memory size of 2MB, or relations of  $< 36\text{ GB}$  with memory of 12 MB



# Handling of Overflows

- Partitioning is said to be **skewed** if some partitions have significantly more tuples than some others
- **Hash-table overflow** occurs in partition  $s_i$  if  $s_i$  does not fit in memory. Reasons could be
  - Many tuples in  $s$  with same value for join attributes
  - Bad hash function
- **Overflow resolution** can be done in build phase
  - Partition  $s_i$  is further partitioned using different hash function.
  - Partition  $r_i$  must be similarly partitioned.



# Cost of Hash-Join

- If recursive partitioning is not required:
  - Partitioning: a complete reading of both relations and a subsequent writing back of them:  $2(b_r + b_s)$  , plus
  - Build and probe phase: reading each of the partitions once:  $b_r + b_s$
  - Total:  $3(b_r + b_s)$  block transfers +  $2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil)$  seeks
  - read/write  $b_b$  blocks at a time
- If recursive partitioning required:
  - number of passes required for partitioning the build relation  $s$  to less than  $M$  blocks per partition is  $\lceil \log_{\lfloor M/b_b \rfloor - 1}(b_s/M) \rceil$
  - best to choose the smaller relation as the build relation.
  - Total cost estimate is:  
$$2(b_r + b_s) \lceil \log_{\lfloor M/b_b \rfloor - 1}(b_s/M) \rceil + b_r + b_s \text{ block transfers} +$$
$$2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil) \lceil \log_{\lfloor M/b_b \rfloor - 1}(b_s/M) \rceil \text{ seeks}$$
- If the entire build input can be kept in main memory no partitioning is required
  - Cost estimate goes down to  $b_r + b_s$ .



# Example of Cost of Hash-Join

*instructor* ⋈ *teaches*

- Assume that memory size is 20 blocks
- $b_{instructor} = 100$  and  $b_{teaches} = 400$ .
- *instructor* is to be used as build input. Partition it into five partitions, each of size 20 blocks. This partitioning can be done in one pass.
- Similarly, partition *teaches* into five partitions, each of size 80. This is also done in one pass.
- $b_b = 3$  blocks
- Therefore total cost, ignoring cost of writing partially filled blocks:
  - $3(100 + 400) = 1500$  block transfers +  
 $2(\lceil 100/3 \rceil + \lceil 400/3 \rceil) = 336$  seeks





# Evaluation of Expressions

- So far: we have seen algorithms for individual operations
- Alternatives for evaluating an entire expression tree
  - **Materialization**: generate results of an expression whose inputs are relations or are already computed, **materialize** (store) it on disk. Repeat.
  - **Pipelining**: pass on tuples to parent operations even as an operation is being executed
- We study above alternatives in more detail

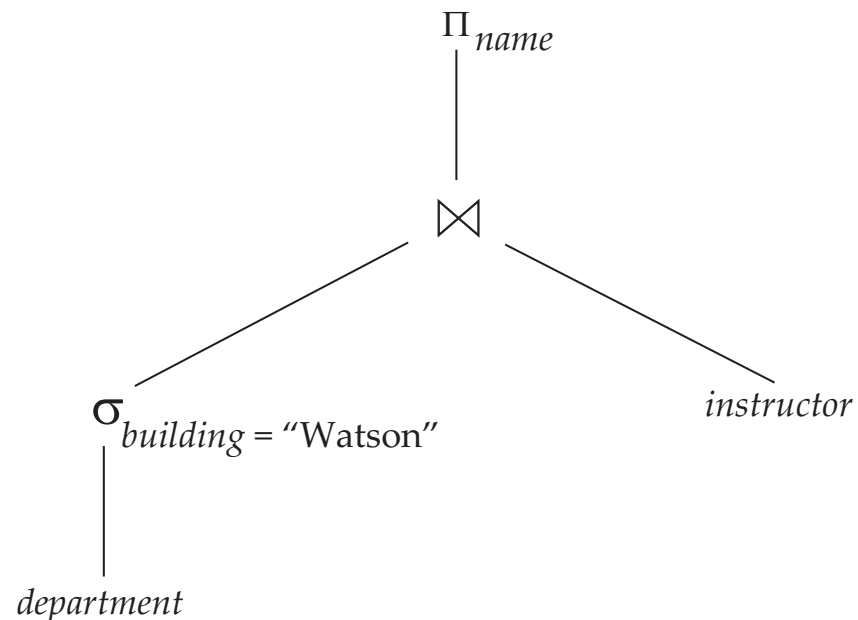


# Materialization

- **Materialized evaluation:** evaluate one operation at a time, starting at the lowest-level. Use intermediate results materialized into temporary relations to evaluate next-level operations.
- E.g., compute and store

$$\sigma_{building="Watson"}(department)$$

then compute and store its join with *instructor*, and finally compute the projection on *name*.





## Materialization (Cont.)

- Materialized evaluation is always applicable
- Cost of writing results to disk and reading them back can be quite high
  - Overall cost = Sum of costs of individual operations + cost of writing intermediate results to disk
- **Double buffering**: use two output buffers for each operation, when one is fully writing it to disk while the other is getting filled
  - Allows overlap of disk writes with computation and reduces execution time



# Pipelining

- **Pipelined evaluation:** evaluate several operations simultaneously, passing the results of one operation on to the next.
- E.g., in previous expression tree, don't store result of

$$\sigma_{building="Watson"}(department)$$

- instead, pass tuples directly to the join. Similarly, don't store result of join, pass tuples directly to projection.
- Much cheaper than materialization: no need to store a temporary relation to disk.
- Pipelining may not always be possible – e.g., sort, hash-join.
- For pipelining to be effective, use evaluation algorithms that generate output tuples even as tuples are received for inputs to the operation.
- Pipelines can be executed in two ways: **demand driven** and **producer driven**



## Pipelining (Cont.)

- In **demand driven** or **lazy** evaluation (**pull** models of pipelining)
  - system repeatedly requests next tuple from top level operation
  - Each operation requests next tuple from children operations as required
  - In between calls, the operation has to maintain “**state**” so it knows what to return next
- In **producer-driven** or **eager** pipelining (**push** models of pipelining)
  - Operators produce tuples eagerly and pass them up to their parents
    - Buffer maintained between operators, child puts tuples in buffer, parent removes tuples from buffer
    - if buffer is full, child waits till there is space in the buffer, and then generates more tuples
  - System schedules operations that have space in output buffer and can process more input tuples



## End of Chapter 15